

```

create database B_M_K;
use B_M_K;

-- SQL Practice Task Sheet :-

-- 1.] Customers Table :-
create table Customers (
    CustomerID int primary key,
    CustomerName varchar(50) not null,
    PhoneNo varchar(10),
    City varchar(30),
    AccountType varchar(20),
    AccountNo int unique
);

insert into Customers
(CustomerID, CustomerName, PhoneNo, City, AccountType, AccountNo)
values
(1, 'Rahul Sharma', '9876543210', 'Pune', 'Savings', 1001),
(2, 'Sneha Patil', '9988776655', 'Mumbai', 'Current', 1002),
(3, 'Aman Verma', '9123456780', 'Nagpur', 'Savings', 1003),
(4, 'Priya Singh', '9012345678', 'Delhi', 'Current', 1004),
(5, 'Karan Mehta', '9871203456', 'Hyderabad', 'Savings', 1005),
(6, 'Neha Joshi', '9988001122', 'Pune', 'Current', 1006),
(7, 'Rohit Kumar', '9765432109', 'Bangalore', 'Savings', 1007),
(8, 'Pooja Sharma', '9876540001', 'Chennai', 'Savings', 1008),
(9, 'Vivek Shah', '9001122334', 'Ahmedabad', 'Current', 1009),
(10, 'Anjali Verma', '9988771100', 'Jaipur', 'Savings', 1010);

-- 2.] Accounts Table :-
create table Accounts (
    AccountID int primary key,
    CustomerID int,
    Balance decimal(12,2),
    OpenDate date,
    foreign key (CustomerID)
    references Customers(CustomerID)
);

insert into Accounts
(AccountID, CustomerID, Balance, OpenDate)
values
(1001, 1, 55000, '2025-01-10'),
(1002, 2, 120000, '2024-11-20'),
(1003, 3, 35000, '2025-03-15'),
(1004, 4, 98000, '2025-02-01'),
(1005, 5, 75000, '2025-01-25'),
(1006, 6, 150000, '2024-12-18'),
(1007, 7, 42000, '2025-04-10'),
(1008, 8, 88000, '2025-05-05'),
(1009, 9, 200000, '2024-09-30'),
(1010, 10, 67000, '2025-03-22');

-- 3.] Transactions Table :-
create table Transactions (
    TransactionID int primary key,
    AccountID int,
    TransactionType varchar(20),

```

```

        Amount decimal(12,2),
        TransactionDate date,
        foreign key (AccountID)
        references Accounts(AccountID)
);

insert into Transactions
(TransactionID, AccountID, TransactionType, Amount, TransactionDate)
values
(1,1001,'Deposit',10000,'2026-06-01'),
(2,1001,'Withdraw',5000,'2026-06-02'),
(3,1002,'Deposit',25000,'2026-06-02'),
(4,1003,'Withdraw',3000,'2026-06-03'),
(5,1004,'Deposit',15000,'2026-06-04'),
(6,1005,'Deposit',12000,'2026-06-05'),
(7,1006,'Withdraw',7000,'2026-06-05'),
(8,1007,'Deposit',9000,'2026-06-06'),
(9,1008,'Withdraw',4500,'2026-06-06'),
(10,1009,'Deposit',30000,'2026-06-07'),
(11,1010,'Withdraw',2000,'2026-06-07'),
(12,1002,'Withdraw',10000,'2026-06-08'),
(13,1003,'Deposit',5000,'2026-06-08'),
(14,1005,'Withdraw',3500,'2026-06-09'),
(15,1007,'Deposit',15000,'2026-06-09');

-- 4.] Loans Table :-
create table Loans (
    LoanID int primary key,
    CustomerID int,
    LoanAmount decimal(12,2),
    LoanType varchar(30),
    foreign key (CustomerID)
    references Customers(CustomerID)
);

insert into Loans
(LoanID, CustomerID, LoanAmount, LoanType)
values
(1,1,500000,'Home Loan'),
(2,2,200000,'Car Loan'),
(3,4,100000,'Education Loan'),
(4,5,300000,'Business Loan'),
(5,6,150000,'Personal Loan'),
(6,8,250000,'Home Loan'),
(7,9,400000,'Business Loan'),
(8,10,180000,'Car Loan');

-- SQL Practice Questions :-

select * from Customers;
select * from Accounts;
select * from Transactions;
select * from Loans;

-- 1.] Display customer names, account numbers, and account balances
using INNER JOIN.

```

```
select c.CustomerName, c.AccountNo, a.Balance
from dbo.Customers c
inner join dbo.Accounts a
on c.CustomerID = a.CustomerID;
```

-- 2.] Find the top 3 customers with the highest account balances.

```
select top(3) c.CustomerName, a.Balance
from dbo.Customers c
inner join dbo.Accounts a
on c.CustomerID = a.CustomerID
order by a.Balance desc;
```

-- 3.] Show all customers who have taken loans along with loan amount and loan type.

```
select c.CustomerName, l.LoanAmount, l.LoanType
from dbo.Customers c
inner join dbo.Loans l
on c.CustomerID = l.CustomerID;
```

-- 4.] Find the total deposited amount and total withdrawn amount separately.

```
select TransactionType, sum(Amount) as TotalAmount
from Transactions
group by TransactionType;
```

-- 5.] Display customer-wise total transaction amount using GROUP BY.

```
select c.CustomerName, sum(t.Amount) as TotalTransactionAmount
from Customers c
inner join Accounts a
on c.CustomerID = a.CustomerID
inner join Transactions t
on a.AccountID = t.AccountID
GROUP BY c.CustomerName;
```

-- 6.] Find customers whose balances are greater than the average bank balance.

```
select c.CustomerName, a.Balance
from Customers c
inner join Accounts a
on c.CustomerID = a.CustomerID
where a.Balance > (select avg(Balance) from Accounts);
```

-- 7.] Show the highest transaction amount performed by each customer.

```
select c.CustomerName, max(t.Amount) as HighestTransaction
from Customers c
inner join Accounts a
on c.CustomerID = a.CustomerID
inner join Transactions t
on a.AccountID = t.AccountID
group by c.CustomerName;
```

-- 8.] Display all customers who have not taken any loans using LEFT JOIN.

```
select c.CustomerName
from Customers c
left join Loans l
on c.CustomerID = l.CustomerID
where l.LoanID is null;
```

-- 9.] Find the total number of transactions performed by each customer.

```
select c.CustomerName, count(t.TransactionID) as TotalTransactions
from Customers c
inner join Accounts a
on c.CustomerID = a.CustomerID
inner join Transactions t
on a.AccountID = t.AccountID
group by c.CustomerName;
```

-- 10.] Rank customers based on their account balances using RANK() window function.

```
select c.CustomerName, a.Balance,
RANK() over (order by a.Balance desc) as BalanceRank
from Customers c
inner join Accounts a
on c.CustomerID = a.CustomerID;
```

-- 11.] Display dense ranking of customers according to balance using DENSE_RANK().

```
select c.CustomerName, a.Balance,
DENSE_RANK() over (order by a.Balance desc) as DenseBalanceRank
from Customers c
inner join Accounts a
on c.CustomerID = a.CustomerID;
```

-- 12.] Show previous transaction amount using LAG() function.

```
select TransactionID, Amount,
LAG(Amount) over (order by TransactionID) as PreviousAmount
from Transactions;
```

-- 13.] Show next transaction amount using LEAD() function.

```
select TransactionID, Amount,
LEAD(Amount) over (order by TransactionID) as NextAmount
from Transactions;
```

-- 14.] Calculate running total of transaction amounts using SUM() OVER().

```
select TransactionID, Amount,
SUM(Amount) over (order by TransactionID) as RunningTotal
from Transactions;
```

```

-- 15.] Find the second highest account balance using subquery or
window function.

-- Using subquery
select max(Balance) as SecondHighestBalance
from Accounts
where Balance < (select max(Balance) from Accounts);

-- Using window function
select Balance as SecondHighestBalance
from
(
    select Balance,
        DENSE_RANK() over(order by Balance DESC) AS raank
    FROM Accounts
) x
WHERE raank = 2;

-- 16.] Find customers who performed more than 2 transactions.

select c.CustomerName, count(t.TransactionID) as TransactionCount
from Customers c
inner join Accounts a
on c.CustomerID = a.CustomerID
inner join Transactions t
on a.AccountID = t.AccountID
group by c.CustomerName
having count(t.TransactionID) >= 2;

-- 17.] Display customer-wise minimum and maximum transaction amounts.

select c.CustomerName,
min(t.Amount) as MinTransaction, max(t.Amount) as MaxTransaction
from Customers c
inner join Accounts a
on c.CustomerID = a.CustomerID
inner join Transactions t
on a.AccountID = t.AccountID
group by c.CustomerName;

```